

Chapter 1: Foundations of Machine Learning

Universidad de Salamanca

Faculty of Sciences

Advanced Programming

Diego M. Jiménez-Bravo · dmjimenez@usal.es

Outline

- 1 Introduction
- 2 1.1 History and evolution
- 3 1.2 Fundamental concepts

1 Introduction

2 1.1 History and evolution

3 1.2 Fundamental concepts

Why learn machine learning?

In **traditional** programming, a person writes explicitly every rule a program must follow. For example:

```
if temperature > 30: turn_on_ac()
```

This works well for simple problems, but many real-world problems are too complex to describe with fixed rules: how do you automatically tell a cat from a dog in a photograph? How do you predict whether a bank transaction is fraudulent? How do you classify emails as spam or not spam?

Machine learning inverts the paradigm: instead of writing the rules by hand, we teach the program to *learn* those rules from examples.

Example: no one can hand-write a rule that tells a cat from a dog in a photo, but a model can *learn* one by seeing thousands of photos already labeled “cat” or “dog”.

Outline

1 Introduction

2 1.1 History and evolution

3 1.2 Fundamental concepts

Definition and the paradigm shift

Artificial Intelligence is concisely defined as the effort to automate intellectual tasks normally performed by humans. It is a broad field that includes machine learning and deep learning, as well as older methods such as **symbolic AI**, which dominated the field from the 1950s until the late 1980s.

The most revolutionary aspect of machine learning is that it represents a **new programming paradigm**: instead of a person writing the rules, the machine analyzes the input data and the expected answers to infer what those rules should be.

For this learning to happen, **three key elements** are required:

- ➊ Input data.
- ➋ Examples of the expected output (**labels**).
- ➌ A way to measure whether the system is doing well (a **loss function**).

Mathematical roots and pioneers (1805-1943)

Although the term “machine learning” is modern, many of its statistical concepts were developed over two centuries ago:

- **1805**: Legendre and Gauss publish the **method of least squares**, the earliest form of linear regression — still the basis of countless models today.
- **1936**: Ronald Fisher proposes **linear discriminant analysis**, one of the first techniques for predicting a category (a qualitative value) from numerical data.
- **1943**: Warren McCulloch and Walter Pitts present the **first computational model of a biological neuron**: a mathematical simplification of how a neuron activates, the seed of today's neural networks.

The Turing Test (1950)

In 1950, Alan Turing published his foundational paper “*Computing Machinery and Intelligence*”, asking: **can machines think?**

To avoid the philosophical debate over what “thinking” means, he proposed a practical experiment known as the **Turing Test**: a person converses in writing with both a human and a machine, without knowing which is which. If they cannot reliably tell them apart, the machine is said to have passed the test.

Although its validity as a true measure of intelligence is debated today, it marked the horizon AI aspired to for decades.



Source: Elliott & Fry (1951), public domain

The formal birth: the Dartmouth workshop (1956)

AI formally crystallized as a research field in **1956**, when John McCarthy organized the **Dartmouth workshop**, a multi-week gathering that brought together the leading researchers of the time.

McCarthy put forward a very ambitious conjecture: every aspect of learning or intelligence could be described so precisely that a machine could simulate it.

For the following decades, most experts believed human-level intelligence would be reached by hand-coding a massive set of **explicit logical rules**. This approach, known as **symbolic AI**, peaked with **expert systems** in the 1980s.

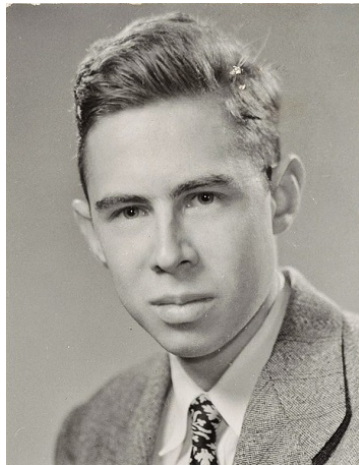
Example: a typical 1980s medical expert system contained thousands of rules of the form “if fever and cough and chest pain, then suspect pneumonia”, hand-written by doctors and programmers.

The Perceptron and the “AI winters”

In **1958**, Frank Rosenblatt created the **Perceptron**, the first artificial-neuron model able to *learn* by adjusting its own parameters from examples, instead of having them hand-programmed.

AI's history has not been steady progress, but a sequence of cycles of extreme enthusiasm followed by disillusionment and funding cuts, known as “**AI winters**”:

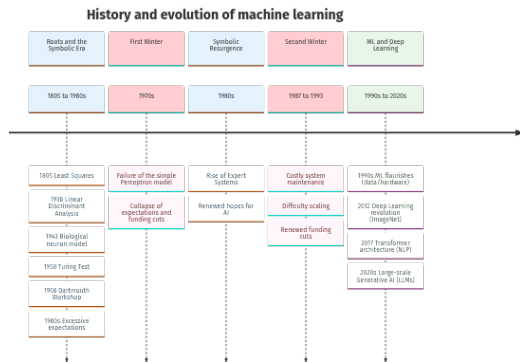
- **First winter** (1970s): it was discovered that the simple Perceptron could not solve problems that were not linearly separable.
- **Second winter** (1990s): expert systems turned out to be expensive to maintain, hard to scale, and limited to very narrow domains.



Source: *Heinz Nixdorf MuseumsForum*,
CC BY-SA 4.0

Full timeline

Machine learning truly took off in the 1990s, driven by faster hardware and the availability of large amounts of data. Unlike symbolic AI —suited to logical problems such as chess—, ML made it possible to tackle “fuzzy” problems such as speech recognition or image classification.

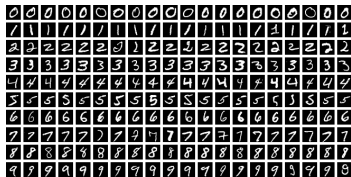


The rise of *Deep Learning* (2010-2016)

Deep Learning is a specialized branch of ML that learns **successive layered representations**: each layer of the neural network receives the output of the previous one and extracts patterns that are increasingly useful for the task.

The turning point came in **2012**, when Geoffrey Hinton's team won the **ImageNet** image-classification challenge by a wide margin, using a deep neural network instead of the traditional statistical methods that had dominated until then.

During this period, the **MNIST** dataset (handwritten digits 0-9) became the quintessential benchmark for comparing classification algorithms.



Source: Suvanjanprasai (2024), Wikimedia Commons, CC BY-SA 4.0

The LLM era (2017-today)

Starting in **2017**, the **Transformer** architecture—which uses an “attention” mechanism to process text sequences without recurrent layers—unleashed a revolution in natural language processing.

This enabled training **foundation models** through **self-supervised learning** on enormous amounts of internet text: the model learns by predicting words it hides from itself, without needing human-created labels.

The current era is marked by **Generative AI** (such as ChatGPT or Gemini), which scales the technology to billions of parameters and allows it not only to classify data, but to **generate** new content: text, images, code, or audio, thanks to **LLMs** (*Large Language Models*).

Outline

- 1 Introduction
- 2 1.1 History and evolution
- 3 1.2 Fundamental concepts**

Data, features, and labels

The success of any ML system depends on how the information is represented. Consider a typical table:

Sample	Feature 1	Feature 2	Feature 3	Label
1	25	70	175	Cat
2	8	4	90	Cat
3	35	80	180	Dog

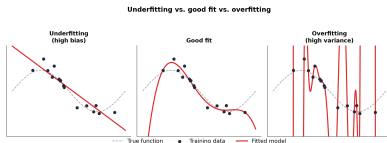
- **Sample:** the basic unit of information (one row of the table), also called an instance or data point.
- **Features:** the attributes describing each sample (the input columns).
- **Label:** the correct answer associated with each sample; the set of all labels makes up the “ground truth”.

Overfitting vs. underfitting

The fundamental problem in machine learning is the tension between fitting the model to known data and its ability to work well on new data:

- **Underfitting**: the model is too simple to capture the real pattern in the data. Both training and validation error are high.
- **Overfitting**: the model is too complex and ends up memorizing the noise in the training data. Training error is very low, but validation error is high.
- The goal is to find the **bias-variance** trade-off: low bias (the model doesn't oversimplify) and low variance (the model isn't overly sensitive to the specific training data).

Example: it's like studying for an exam by memorizing the exact answers of one mock test (overfitting) instead of understanding the topic: you'll fail as soon as the questions change a little.

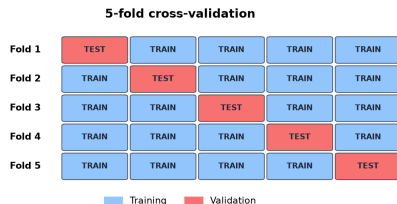


K-fold cross-validation

To measure generalization reliably, it is not enough to evaluate the model on the same data it was trained on. The standard practice is to split the data into three sets:

- **Training:** to fit the model's weights.
- **Validation:** to choose hyperparameters and compare models.
- **Test:** reserved for a final, unbiased evaluation.

When data is scarce, K -fold cross-validation makes better use of the information: the data is split into K parts, the model is trained K times (each time using a different part as validation), and the final metric is the **average** of the K results. **Stratification** ensures each partition keeps the same class proportions as the original dataset.



Confusion matrix

To evaluate a classifier we use a table called the **confusion matrix**, which compares the true labels with the model's predictions:

Confusion matrix

		Prediction	
		Positive	Negative
Actual class	Positive	TP True Positive	FN False Negative
	Negative	FP False Positive	TN True Negative

- **TP** (true positive): a correct positive.
- **TN** (true negative): a correct negative.
- **FP** (false positive): a “false alarm” (type I error).
- **FN** (false negative): a “miss” (type II error).

Example: in a medical diagnosis, a false negative (telling a sick person “healthy”) is usually far more serious than a false positive.

Metrics derived from the confusion matrix

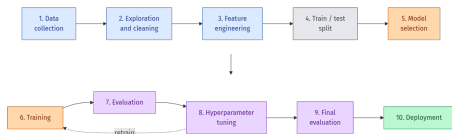
- **Accuracy**: the percentage of correct predictions overall. Can be misleading when classes are imbalanced.
- **Precision**: of everything the model predicted as positive, how much was actually correct?
- **Recall** (sensitivity): of all the real positives out there, how many did the model find?
- **F1**: combines precision and recall into a single number; useful when both matter equally.
- **AUC-ROC**: measures how well the model tells positives and negatives apart, without depending on a fixed threshold. 1.0 is perfect, 0.5 is equivalent to chance.

Example: if 90% of emails aren't spam, a model that always says "not spam" would have 90% accuracy without having learned anything useful: that's why precision and recall sometimes matter more than accuracy alone.

The ML *pipeline*: 10 stages

Developing a machine learning project follows a universal workflow, from data collection to production deployment. It is an **iterative** process: the evaluation stage often forces a return to earlier stages (for example, collecting more data or tuning hyperparameters) before the project can be considered finished.

- 1 Data collection
- 2 Exploration and cleaning
- 3 Feature engineering
- 4 Train/test split
- 5 Model selection
- 6 Training
- 7 Evaluation
- 8 Hyperparameter tuning
- 9 Final evaluation
- 10 Deployment (and continuous monitoring)



Chapter summary

- Machine learning was born from a very simple question: **can machines learn?**
- Its history has not been linear: it has gone through cycles of hype and “winters” of disillusionment and funding cuts.
- The current state of the art is **deep learning** and **LLMs**, made possible by more data, better hardware, and more efficient algorithms.
- **Sample**, **feature**, and **label** are the basic building blocks of any dataset.
- **Overfitting/underfitting** and **cross-validation** are the tools for knowing whether a model generalizes well.
- The **confusion matrix** and its derived metrics let you pick the right success measure for the problem at hand.
- The ML **pipeline** organizes the whole process into 10 iterative stages, from data to deployment.

This material is distributed under a
Creative Commons Attribution-NonCommercial 4.0
(CC BY-NC 4.0)

You are free to:

- **Share:** copy and redistribute the material in any medium or format.
- **Adapt:** remix, transform and build upon the material.

For example: use them in class, upload them to a virtual campus, translate them or reuse a figure in your own notes.

Under the following terms:

- **Attribution (BY):** you must credit the author, link to the license and indicate if changes were made.
- **NonCommercial (NC):** you may not use the material for commercial purposes.

You may not add restrictions that legally prevent others from doing anything the license permits.

creativecommons.org/licenses/by-nc/4.0/

Chapter 1: Foundations of Machine Learning

Universidad de Salamanca

Faculty of Sciences

Advanced Programming

Diego M. Jiménez-Bravo · `dmjimenez@usal.es`